

Method:

I chose to use a basic depth-first search approach for this problem because this is essentially a graph problem.

Approach:

This solution uses a recursive dfs implementation that keeps track of visited rooms and then returns true if all rooms are visit-able. The algorithm takes in a list of key sets that represents the rooms and keys, calling a recursive version that current room to the visited list, then visits each room that can be unlocked with a key from that room in the same way.

Reasoning:

This algorithm is correct because it will be able to explore anything that it can get a key for. This is true because it will go down from room 0, into each room that 0 contains a key for, and then into each room from there in the same way. It works because this problem can be broken down into a depth-first search where each room is a node and each key is an edge between two rooms. This program will return true if the graph is connected and false if it's disconnected.

Reflection:

I used these two cases to test for correctness:

rooms = [[1],[2],[3],[]]

rooms = [[1,3],[3,0,1],[2],[0]]

Time Complexity:

This algorithm is $O(\text{rooms} + \text{keys})$